

Midterm Mastery (Part I)

Weifan Li

1. Thinking in Trade-offs: CAP Theorem & FLP Impossibility

The most transformative concept for me has been learning that distributed systems design is fundamentally about choosing which problems you are willing to live with. The CAP theorem initially seemed like a simple "pick two out of three" rule, but the readings made clear that this framing is oversimplified — partitions are inevitable, so the real question is: when a partition happens, do you sacrifice consistency or availability?

This clicked when I reflected on my own e-commerce project, Mountify. I use Redis as a caching layer in front of PostgreSQL, which inherently creates a window where the cache and database can disagree — a form of eventual consistency. Reading about CAP formalized why my early attempts to make this "perfectly" consistent had diminishing returns: for a shopping platform, briefly stale inventory data is acceptable, but blocking all reads until every layer agrees would destroy performance. The FLP impossibility result reinforced this: even under ideal conditions, deterministic consensus in an asynchronous system is provably impossible. You must design around uncertainty, not try to eliminate it.

2. MapReduce: The Power of Constrained Abstractions

Reading the original MapReduce paper was a highlight. I appreciated how restricting the programming model to just two operations — map and reduce — enables automatic parallelization, fault tolerance, and data locality optimization. The backup task mechanism (Section 3.6) was especially clever: instead of preventing stragglers, just run redundant tasks near completion, yielding a 44% speedup in the sort benchmark. My critical takeaway was that the paper does not adequately discuss the performance cost of converting Google's indexing system into eight sequential MapReduce passes, each writing intermediate results to disk. The trade-off between developer productivity and raw performance is real, and modern systems like Spark address this with in-memory processing.

3. From Bottleneck to Auto Scaling: Hands-on with HW6

Homework 6 was where theory met practice most directly. In Part II, load testing a Go product search service on ECS Fargate revealed a textbook CPU-bound bottleneck — CPU scaled linearly with concurrency while memory stayed flat. In Part III, adding an ALB and Auto Scaling policy resolved the bottleneck dynamically: under 500 concurrent users, the system scaled from 2 to 4 instances within 2 minutes, maintaining stable response times throughout.

4. Infrastructure as Code & Docker

Terraform changed how I think about cloud deployments. Going from 20 clicks in the AWS console (HW1) to a single *terraform apply* command that reproducibly provisions EC2 instances, security groups, ALBs, and Auto Scaling policies was a paradigm shift. Docker, especially multi-stage builds, was another key learning. Reducing a Go image from 1 GB (with the full toolchain) to 30 MB (just the binary in a distroless base) drove home the principle of minimizing the attack surface. These tools together enable the reproducibility and portability that distributed systems demand.

5. Parnas, Information Hiding & Microservices

Parnas's 1972 paper on decomposing systems into modules was surprisingly relevant to modern microservice architecture. His information hiding principle maps directly onto how microservices are

designed today: each service owns its data and internal logic, exposing only a well-defined API. His argument that modules should encapsulate design decisions likely to change anticipated the single responsibility principle and independent deployability that microservices rely on. It was striking to see the theoretical foundation of modern distributed architectures laid out so clearly over 50 years ago.

6. Distributed Systems Fundamentals

The foundational readings on failure models (halting, omission, Byzantine), the 8 Fallacies of Distributed Computing, and RPC semantics gave me a vocabulary for problems I had already encountered but could not name. For example, when deploying Mountify to a VPS, I hit cold-start latency issues because I assumed "latency is zero" — one of the classic fallacies. Understanding that distributed systems must be designed assuming failure happens all the time (Ken Arnold's principle) reframed how I approach every architectural decision, from retry logic to health checks to the ALB configuration in HW6.

Overall, the most valuable thread running through this course is learning to reason about trade-offs under uncertainty. Whether it is CAP, MapReduce's simplicity-vs-performance, or vertical-vs-horizontal scaling, the recurring lesson is: there is no perfect solution, only informed choices. The hands-on assignments — especially the progression from a single EC2 instance to a fully auto-scaled ECS cluster — made these abstract ideas concrete and actionable.